GPUMODE Lecture Study Notes: Lecture 60 Optimizing Linear Attention

Core Problem the Lecture Addresses

This lecture studies how to make **linear attention** and related recurrent sequence models train and run efficiently on modern accelerators, especially GPUs. The main speaker, Songlin Yang, focuses on a family of architectures that replace quadratic softmax attention with recurrent associative-memory mechanisms. The goal is not merely to reduce asymptotic complexity, but to design sequence layers whose mathematical form maps well to GPU hardware.

The central problem is the following:

> Softmax attention has excellent in-context retrieval because it stores and directly accesses key-value pairs through the KV cache, but it has quadratic training cost and linearly growing inference memory. Linear attention removes the KV cache from first principles, but naive recurrent implementations are sequential, tensor-core unfriendly, and often weak at in-context retrieval. The challenge is to recover both hardware efficiency and retrieval quality.

The lecture therefore addresses three tightly coupled questions:

1. How can linear attention be rewritten so that training uses **matrix multiplication** rather than rank-one recurrent updates?
2. How can chunkwise algorithms exploit the associativity of linear attention to obtain sequence-level parallelism without materializing an enormous recurrent state at every token?
3. How can architectures such as **DeltaNet**, **Gated DeltaNet**, **test-time training layers**, and **Mesa-like layers** improve the associative recall limitations of vanilla linear attention?

The hardware story is especially important. Modern NVIDIA GPUs are extremely good at dense matrix multiplication because tensor cores provide high throughput for structured GEMM-like workloads. They are much less efficient for long sequential loops of rank-one outer products and matrix-vector multiplies.

Linear attention therefore becomes practically useful only when the recurrence is transformed into chunk-level matrix operations.

Required Background

Standard Softmax Attention

For a single attention head, assume query, key, and value matrices

$$Q, K, V \in \mathbb{R}^{L \times d},$$

where L is the sequence length and d is the head dimension. In causal self-attention, the output is

$$O = \mathrm{softmax}\left(QK^\top + M\right) V,$$

where M is a causal mask that prevents position t from attending to future positions.

At a single decoding step t, autoregressive softmax attention can be written as

$$o_t = \sum_{j=1}^{t} \alpha_{tj} v_j, \qquad \alpha_{tj} = \frac{\exp(q_t^\top k_j)}{\sum_{\ell=1}^{t} \exp(q_t^\top k_\ell)}.$$

Here:

- $q_t \in \mathbb{R}^d$ is the query at step t.
- $k_j \in \mathbb{R}^d$ and $v_j \in \mathbb{R}^d$ are historical key and value vectors.
- α_{tj} is a normalized attention weight.

The training cost of full attention is dominated by forming and applying the $L \times L$ attention matrix:

$$\text{Training cost} = O(L^2 d).$$

The inference memory cost is dominated by the KV cache:

$$\text{KV cache size per layer per head} = O(Ld).$$

For long-context workloads such as video generation, DNA modeling, RNA modeling, and long-form generation, both costs become serious bottlenecks.

GPU Execution Model

A GPU program is launched as a **grid** of thread blocks. Each block is scheduled onto a streaming multiprocessor, or **SM**. Threads are executed in groups of 32 called **warps**. Important memory and compute resources include:

- **Global memory**: large high-bandwidth memory, usually HBM, but with high latency.
- **L2 cache**: shared cache across SMs.
- **Shared memory**: programmer-managed memory inside each SM.
- **Registers**: fastest per-thread storage.
- **Tensor cores**: specialized matrix-multiply units optimized for FP16, BF16, TF32, FP8, INT8, and related formats.

A kernel is usually fast when it:

- has enough parallelism to occupy many SMs,
- performs coalesced global memory accesses,
- reuses data from registers or shared memory,
- maps arithmetic to tensor-core GEMM operations,
- avoids excessive synchronization and memory traffic,
- maintains a good balance between occupancy and register usage.

Why GEMM Matters

A matrix multiplication

$$C = AB, \qquad A \in \mathbb{R}^{m\times k}, \quad B \in \mathbb{R}^{k\times n}, \quad C \in \mathbb{R}^{m\times n},$$

has high arithmetic intensity because each loaded element of A and B can be reused many times. A rank-one outer product update,

$$S_t = S_{t-1} + k_t v_t^\top,$$

does only $O(d^2)$ operations per token but has a poor shape for tensor cores when executed one token at a time. A matrix-vector readout,

$$o_t = q_t^\top S_t,$$

is also less tensor-core friendly than a matrix-matrix multiply.

Thus, the optimization target is to turn many small sequential operations into larger block-level matrix multiplications.

Arithmetic Intensity and Roofline Thinking

Arithmetic intensity is

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

A workload is roughly **memory-bound** if its arithmetic intensity is too low to saturate compute units, and **compute-bound** if it has enough reuse to saturate arithmetic units. The roofline model can be written as

$$\text{Achievable FLOP/s} = \min\left(\text{Peak FLOP/s}, \text{Memory Bandwidth} \times \text{Arithmetic Intensity}\right).$$

Linear attention optimization is largely about increasing arithmetic intensity by using chunked matrix multiplications and avoiding unnecessary HBM materialization.

Main Concepts

Vanilla Linear Attention

The lecture begins by removing the softmax operator from causal attention. Ignoring normalization variants, linear attention computes

$$o_t = \sum_{j=1}^{t} (q_t^\top k_j) v_j.$$

Because scalar multiplication is associative, this can be rearranged as

$$o_t = q_t^\top \left(\sum_{j=1}^{t} k_j v_j^\top \right).$$

Define a recurrent hidden state

$$S_t = \sum_{j=1}^{t} k_j v_j^\top \in \mathbb{R}^{d \times d}.$$

Then linear attention becomes the recurrence

$$S_t = S_{t-1} + k_t v_t^\top,$$

$$o_t = q_t^\top S_t.$$

This is the key first-principles change: instead of storing all historical keys and values in a KV cache, the model stores a fixed-size matrix-valued recurrent memory S_t.

State Expansion

Classical recurrent neural networks often use a hidden state of size $O(d)$. Linear attention uses a state of size

$$\dim(S_t) = d \times d = O(d^2).$$

This is a form of **state expansion**. The recurrent state has much greater capacity than a vector hidden state, while still being independent of sequence length L. At inference time, the memory footprint per layer per head is

$$O(d^2),$$

rather than

$$O(Ld).$$

This is attractive for long generation, where L may be very large.

Parallel Form versus Recurrent Form

Linear attention has at least two natural forms.

Parallel Form

For full sequence training, one can write

$$O = \left(QK^\top \odot M_{\text{causal}}\right) V,$$

where M_{causal} masks out future tokens. This resembles softmax attention without the softmax. It is parallel across sequence positions, but it still forms an $L \times L$ attention matrix. Its complexity is

$$O(L^2 d).$$

Thus, it does not solve the long-context training bottleneck.

Recurrent Form

The recurrent form is

$$S_t = S_{t-1} + k_t v_t^\top, \qquad o_t = q_t^\top S_t.$$

This avoids the $L \times L$ matrix and has sequential complexity

$$O(Ld^2).$$

However, it has two hardware problems:

1. It is sequential over t, limiting parallelism along the sequence dimension.
2. It uses rank-one outer products and matrix-vector multiplies, which do not exploit tensor cores as effectively as matrix-matrix multiplication.

Why Parallel Scan Alone Is Not Enough

A natural idea is to use a parallel prefix scan, since

$$S_t = \sum_{j=1}^{t} k_j v_j^\top$$

is an associative cumulative sum. Parallel scan can expose sequence-level parallelism. However, the lecture emphasizes that this is not enough.

If a scan materializes S_t for every token, the memory complexity becomes

$$O(Ld^2).$$

This is much larger than storing Q, K, V, whose total size is

$$O(Ld).$$

For large d, materializing all states is unacceptable. Also, scan-based computation still does not naturally convert the core update into tensor-core GEMMs unless carefully blocked.

Chunkwise Linear Attention

The chunkwise form sits between the recurrent and parallel forms. Let the sequence be divided into chunks of length C. For chunk index i, define

$$Q_i, K_i, V_i, O_i \in \mathbb{R}^{C \times d}.$$

The chunkwise algorithm computes recurrent state only at chunk boundaries rather than every token. If S_i denotes the state at the end of chunk i, then

$$S_i = S_{i-1} + K_i^\top V_i.$$

This works because

$$K_i^\top V_i = \sum_{r=1}^{C} k_{i,r} v_{i,r}^\top,$$

where $k_{i,r}$ and $v_{i,r}$ are the key and value at local position r in chunk i. The sum of outer products is now expressed as a matrix multiplication.

For the output inside chunk i, there are two contributions:

$$O_i = O_i^{\text{history}} + O_i^{\text{local}}.$$

The historical contribution uses the previous chunk state:

$$O_i^{\text{history}} = Q_i S_{i-1}.$$

The local contribution uses causal linear attention within the current chunk:

$$O_i^{\text{local}} = \left(Q_i K_i^\top \odot M_C\right) V_i,$$

where M_C is a $C \times C$ causal mask local to the chunk.

Thus,

$$O_i = Q_i S_{i-1} + \left(Q_i K_i^\top \odot M_C\right) V_i.$$

This formula is mathematically equivalent to full causal linear attention. It is not an approximation.

Complexity of Chunkwise Linear Attention

There are L/C chunks. Each state update costs

$$O(Cd^2)$$

because $K_i^\top V_i$ multiplies a $d \times C$ matrix by a $C \times d$ matrix. Across all chunks:

$$\text{State-update cost} = O(Ld^2).$$

The local chunk attention cost per chunk is

$$O(C^2 d),$$

and across all chunks:

$$\text{Local-attention cost} = O\left(\frac{L}{C} C^2 d\right) = O(LCd).$$

Therefore, the total cost is

$$O(Ld^2) + O(LCd).$$

When C is a fixed hardware-friendly constant such as 64, 128, or 256, the complexity is linear in L:

$$O(Ld^2 + LCd) = O(L).$$

The constant factors depend strongly on how well the kernels use tensor cores and memory bandwidth.

Flash Linear Attention

The lecture describes Flash Linear Attention as an IO-aware implementation of chunkwise linear attention. The main idea is analogous to FlashAttention: fuse kernels and avoid materializing intermediate matrices in HBM.

Important reuse opportunities include:

- Q_i is used for both historical readout $Q_i S_{i-1}$ and local chunk attention.
- K_i is used for both the state update $K_i^\top V_i$ and local attention $Q_i K_i^\top$.
- V_i is used for both the state update and local output computation.

A naive PyTorch implementation may materialize intermediate tensors and repeatedly read the same blocks from HBM. A fused kernel can keep tiles in registers or shared memory, perform several subcomputations, and write only the final required outputs. The lecture mentions that such fusion can produce large speedups, around several times faster than a straightforward PyTorch implementation.

Associative Memory View

Linear Attention as Key-Value Associative Memory

Linear attention stores key-value associations through

$$S_t = \sum_{j=1}^{t} k_j v_j^\top.$$

To retrieve the value associated with a query key q, the model computes

$$o = q^\top S_t = \sum_{j=1}^{t} (q^\top k_j) v_j.$$

If $q = k_m$, then

$$o = (k_m^\top k_m) v_m + \sum_{\substack{j=1 \\ j \neq m}}^{t} (k_m^\top k_j) v_j.$$

The desired term is

$$(k_m^\top k_m) v_m,$$

and the unwanted retrieval error is

$$e_m = \sum_{\substack{j=1 \\ j \neq m}}^{t} (k_m^\top k_j) v_j.$$

For perfect retrieval, one would like

$$k_m^\top k_j = 0 \qquad \text{for all } j \neq m.$$

That is, keys should be mutually orthogonal. But in $\mathbb{R}^d$, there can be at most d mutually orthogonal nonzero vectors. Once the number of stored associations exceeds the dimension, cross terms are unavoidable.

This explains why vanilla linear attention can struggle with in-context retrieval. Increasing head dimension helps because it creates more room for approximately orthogonal keys, but this is a partial solution, not a complete fix.

Multi-Query Associative Recall

The lecture discusses a synthetic benchmark called multi-query associative recall. The task presents a context containing key-value pairs, such as uppercase letters mapped to numbers. Later, the model receives query keys and must output the associated values.

The important property is that the answer is provided only in the prompt context. The model cannot memorize it through static parameters because examples are generated on the fly. Therefore, the task directly tests in-context associative memory.

Softmax transformers solve this easily because the KV cache explicitly stores past keys and values. Many recurrent or subquadratic models, such as Mamba-style or gated recurrent architectures, can struggle when model dimension is small. DeltaNet performs better because it has a memory-update mechanism that can erase outdated associations and write new ones.

DeltaNet

Motivation

Vanilla linear attention only adds new associations:

$$S_t = S_{t-1} + k_t v_t^\top.$$

It does not explicitly remove or correct an old value associated with the same key. DeltaNet introduces a more dynamic memory-management rule. The model first retrieves the old value associated with the current key, computes a difference between the new value and the old value, and writes that difference into memory.

DeltaNet Update Rule

Let

$$\beta_t \in (0, 1)$$

be a data-dependent scalar generated by a linear projection followed by a sigmoid. It can be interpreted as a writing strength or learning rate.

Given key k_t, value v_t, and state S_{t-1}, first retrieve the old value:

$$\hat{v}_t = S_{t-1}^\top k_t.$$

Then compute the value residual:

$$\Delta v_t = v_t - \hat{v}_t.$$

The memory update is

$$S_t = S_{t-1} + \beta_t k_t \Delta v_t^\top.$$

Equivalently,

$$S_t = S_{t-1} + \beta_t k_t \left(v_t - S_{t-1}^\top k_t\right)^\top.$$

Expanding the update,

$$S_t = S_{t-1} + \beta_t k_t v_t^\top - \beta_t k_t k_t^\top S_{t-1}.$$

This can be rewritten as

$$S_t = \left(I - \beta_t k_t k_t^\top\right) S_{t-1} + \beta_t k_t v_t^\top.$$

Define the transition matrix

$$A_t = I - \beta_t k_t k_t^\top.$$

Then

$$S_t = A_t S_{t-1} + \beta_t k_t v_t^\top.$$

This form is important because DeltaNet is no longer a pure cumulative sum. It has a structured state transition matrix that can erase or attenuate old information along the direction k_t.

Interpretation of β_t

The scalar β_t controls how strongly the current key-value association modifies memory:

$$\beta_t \approx 0 \quad \Rightarrow \quad S_t \approx S_{t-1},$$

$$\beta_t \approx 1 \quad \Rightarrow \quad \text{strong overwrite or correction along } k_t.$$

From the associative-memory perspective, β_t is a write gate. From the online-learning perspective, it is a learning rate.

DeltaNet as Online Linear Regression

DeltaNet can be derived from a test-time or online-learning objective. Treat the recurrent state S as a fast weight matrix. For a key-value pair (k_t, v_t), define a linear regression loss

$$\mathcal{L}_t(S) = \frac{1}{2} \left\| S^\top k_t - v_t \right\|_2^2 .$$

The gradient with respect to S is

$$\nabla_S \mathcal{L}_t = k_t \left(S^\top k_t - v_t \right)^\top .$$

A gradient descent step with learning rate β_t gives

$$S_t = S_{t-1} - \beta_t \nabla_S \mathcal{L}_t(S_{t-1}),$$

so

$$S_t = S_{t-1} - \beta_t k_t \left(S_{t-1}^\top k_t - v_t \right)^\top .$$

This becomes

$$S_t = S_{t-1} + \beta_t k_t \left(v_t - S_{t-1}^\top k_t \right)^\top ,$$

which is exactly the DeltaNet update.

> DeltaNet is linear attention where the recurrent memory performs an online regression update: retrieve the old value, measure the prediction error, and write the correction.

Why the Online Objective Makes Sense

In softmax attention, the model stores key-value pairs directly in the KV cache. A recurrent model has no KV cache. Therefore, we want the recurrent state itself to encode key-value associations accurately.

The objective

$$\mathcal{L}_t(S) = \frac{1}{2} \left\| S^\top k_t - v_t \right\|_2^2$$

directly trains the fast state to predict values from keys. This is aligned with in-context retrieval: given a key from the prompt, the state should return the correct value.

Hardware-Efficient DeltaNet Training

Unrolling the DeltaNet Recurrence

DeltaNet has the recurrence

$$S_t = A_t S_{t-1} + B_t,$$

where

$$A_t = I - \beta_t k_t k_t^\top, \qquad B_t = \beta_t k_t v_t^\top.$$

Unrolling gives

$$S_t = A_t A_{t-1} \cdots A_1 S_0 + \sum_{j=1}^{t} \left(A_t A_{t-1} \cdots A_{j+1} \right) B_j.$$

For arbitrary dense $A_t \in \mathbb{R}^{d \times d}$, this would be very expensive. DeltaNet is tractable because each transition matrix is structured:

$$A_t = I - \beta_t k_t k_t^\top.$$

This is an identity plus a rank-one update. When $\beta_t = 2$ and k_t is normalized, this resembles a Householder transformation. Products of Householder-like matrices can be represented efficiently using classical numerical linear algebra techniques.

WY Representation

The lecture refers to the WY representation, a compact way to represent products of Householder-like transformations. Instead of explicitly multiplying many $d \times d$ matrices, one can represent their product in a lower-rank form.

For a block of transformations, the cumulative product can be written abstractly as

$$A_C A_{C-1} \cdots A_1 = I - W Y^\top,$$

where $W, Y \in \mathbb{R}^{d \times C}$ are block matrices computed from the keys and gates within the chunk.

The exact construction depends on the DeltaNet parameterization, but the key computational benefit is:

> A cumulative product of structured transition matrices is transformed into operations involving matrix multiplications, triangular solves, and cumulative sums over a chunk.

This makes DeltaNet compatible with the same chunkwise hardware philosophy as linear attention.

Chunkwise DeltaNet

For a chunk of length C, DeltaNet needs to compute:

1. the transition product across the chunk,
2. the transformed contribution of all key-value updates inside the chunk,
3. the output for each token in the chunk,
4. the next chunk-boundary state.

At a high level, the chunkwise state update has the form

$$S_i = \bar{A}_i S_{i-1} + \bar{B}_i,$$

where

$$\bar{A}_i = A_{i,C} A_{i,C-1} \cdots A_{i,1}$$

is the product of transition matrices inside chunk i, and $\bar{B}_i$ aggregates all transformed value updates inside the chunk.

The main computational objective is to express $\bar{A}_i$ and $\bar{B}_i$ using GEMM-like operations.

Triangular Structure Inside a Chunk

Inside a causal chunk, token r may depend only on earlier local tokens $1, \dots, r$. This naturally creates lower-triangular matrices. If

$$T \in \mathbb{R}^{C \times C}$$

is a lower-triangular matrix describing within-chunk dependency, then one often needs operations such as

$$X = T^{-1}Y$$

or

$$X = TY.$$

For small fixed C, such triangular solves can be efficient. The lecture mentions that forward substitution can compute the inverse action of the lower-triangular matrix, after which matrix multiplications can produce the required W-like and U-like block quantities.

The important hardware point is that C is chosen small enough that $C \times C$ matrices are manageable, while large enough to expose tensor-core-friendly work.

Why Larger Hidden Dimension Increases Speedup

The lecture reports that chunkwise DeltaNet becomes increasingly advantageous as head dimension d and sequence length L grow.

This is expected because the recurrent form performs sequential rank-one and matrix-vector operations. As d grows, the number of FLOPs grows, but the operation shape remains tensor-core unfriendly. Chunkwise computation rewrites the work as matrix multiplication, so larger d means more useful tensor-core work.

The speedup also grows with sequence length because long sequences reduce the available batch dimension under a fixed token budget. If a GPU processes a fixed number of tokens per mini-batch, then increasing L typically decreases batch size B:

$$B \approx \frac{\text{tokens per batch}}{L}.$$

When B is large, batch and head dimensions may provide enough parallelism. When L is large and B is small, a purely recurrent implementation lacks enough parallel work. Chunkwise methods expose parallelism along the sequence dimension.

This is analogous to the motivation behind FlashAttention-2, where splitting work across sequence blocks becomes important when batch size times number of heads is too small to fill all SMs.

Gated DeltaNet

The lecture briefly discusses Gated DeltaNet, which combines DeltaNet's transition matrix with a Mamba-2-like gating mechanism. A generic gated update can be understood as

$$S_t = g_t \odot \left(A_t S_{t-1} + \beta_t k_t v_t^\top \right),$$

or more generally as a state-space-style gating mechanism applied to the DeltaNet memory update. The exact implementation may vary, but the conceptual goal is to combine:

- DeltaNet's ability to correct key-value associations,
- Mamba-style gating and decay for stable long-range sequence modeling,
- chunkwise tensor-core-friendly training.

The practical point is that memory update quality and hardware efficiency must be co-designed. A model that is expressive but cannot be efficiently trained is not useful at scale; a model that is fast but forgets in-context information is also insufficient.

Test-Time Training Layers

Fast Weights

The lecture connects DeltaNet to the broader test-time training framework. In this view, the recurrent state is a **fast weight**: a parameter-like object that changes during sequence processing.

Slow weights are the ordinary neural network parameters learned during training. Fast weights are updated at test time based on the current sequence context.

For DeltaNet, the fast weight is

$$S_t \in \mathbb{R}^{d \times d}.$$

It is updated by an online gradient step on a linear regression objective.

From Linear Regression to Nonlinear Regression

DeltaNet assumes a linear prediction model:

$$\hat{v}_t = S^\top k_t.$$

Test-time training layers generalize this by using a nonlinear predictor

$$\hat{v}_t = f_{\theta_t}(k_t),$$

where θ_t is a fast weight updated during sequence processing. The objective becomes

$$\mathcal{L}_t(\theta) = \frac{1}{2} \left\| f_\theta(k_t) - v_t \right\|_2^2 .$$

The fast weights are updated by

$$\theta_t = \theta_{t-1} - \eta_t \nabla_\theta \mathcal{L}_t(\theta_{t-1}).$$

If f_θ is nonlinear, this recurrence is generally nonlinear and no longer easy to parallelize across the sequence dimension.

TTT-Linear

A TTT-linear layer is close to DeltaNet, but it may include additional normalization or nonlinear output transformations. A simplified representation is

$$f_S(k) = \phi(S^\top k),$$

where ϕ may include layer normalization or another nonlinearity. This increases expressivity beyond pure linear associative memory.

TTT-MLP

A TTT-MLP layer uses a small neural network as the fast-weight model:

$$f_\theta(k) = W_2 \sigma(W_1 k),$$

where the fast weights may include some or all of W_1, W_2. This is more expressive than DeltaNet, but the update becomes more expensive and harder to parallelize.

Mini-Batch Gradient Descent in TTT

DeltaNet uses online gradient descent, updating after every token. TTT layers often use mini-batch gradient descent over chunks. For a chunk of size C, the objective may be

$$\mathcal{L}_{\text{chunk}}(\theta) = \frac{1}{2C} \sum_{r=1}^{C} \|f_\theta(k_r) - v_r\|_2^2 .$$

The update is

$$\theta_{\text{next}} = \theta_{\text{prev}} - \eta \nabla_\theta \mathcal{L}_{\text{chunk}}(\theta_{\text{prev}}).$$

This enables multiple tokens in a chunk to be processed together, improving hardware efficiency. However, treating tokens inside a chunk as independent training examples weakens intra-chunk sequential dependency.

To compensate, TTT methods may combine:

- inter-chunk recurrent updates,
- intra-chunk lightweight recurrence,
- local attention within chunks.

Titans-Style Optimizer Improvements

The lecture mentions follow-up work that improves the optimizer used for fast weights. Instead of vanilla SGD, one can use momentum and weight decay:

$$m_t = \mu m_{t-1} + \nabla_\theta \mathcal{L}_t(\theta_{t-1}),$$

$$\theta_t = (1 - \lambda)\theta_{t-1} - \eta m_t.$$

Here:

- m_t is a momentum state,
- μ is the momentum coefficient,
- λ is a weight decay coefficient,
- η is a learning rate.

The motivation is that if test-time state updates are viewed as optimization, then better optimizers can produce better fast-memory behavior.

Large-Chunk TTT and TTT-Downright

Chunk Size Trade-Off

In TTT-style methods, increasing chunk size improves hardware efficiency because more work can be batched into larger matrix operations. But larger chunks reduce the frequency of recurrent state updates:

$$\text{Update frequency} = \frac{1}{C}.$$

When C is large, the recurrent memory is updated less often, which can hurt modeling quality.

Using Chunk Softmax Attention

The lecture discusses a workaround: use local chunk softmax attention while letting the recurrent state update less frequently. The local attention handles short-range dependencies inside the chunk, while the recurrent memory focuses on historical information outside the chunk.

A hybrid large-chunk architecture may compute

$$O_i = \text{LocalAttention}(Q_i, K_i, V_i) + \text{RecurrentMemoryRead}(Q_i, S_{i-1}).$$

This lets the chunk size C be much larger without losing all intra-chunk dependency. It also allows simpler and more efficient implementations, sometimes even in pure PyTorch, because the state update is less frequent.

GPU Throughput Implication

Small-chunk TTT with nonlinear fast weights can be slow on GPUs because it requires frequent updates and complicated kernels. Large-chunk TTT shifts more work into conventional batched operations, improving throughput. The lecture notes that TTT variants can be fast on TPUs but slow on GPUs unless the computation is carefully reshaped.

Mesa-Like Layers and Recursive Least Squares

Limitation of Single-Pair Objectives

DeltaNet and basic TTT objectives optimize the current key-value pair:

$$\mathcal{L}_t(S) = \frac{1}{2} \left\| S^\top k_t - v_t \right\|_2^2.$$

This does not directly optimize retrieval over the entire history. A more informed objective includes all past key-value pairs.

History-Aware Objective

A history-aware online objective can be written as

$$\mathcal{L}_t(S) = \frac{1}{2}\sum_{j=1}^{t} \gamma^{t-j} \left\| S^\top k_j - v_j \right\|_2^2 + \frac{\lambda}{2}\|S\|_F^2,$$

where:

- $\gamma \in (0, 1]$ is a forgetting factor,
- recent pairs receive larger weight when $\gamma < 1$,
- λ regularizes the recurrent state norm.

This resembles recursive least squares. In closed form, if

$$K_t = \begin{bmatrix} k_1^\top \\ k_2^\top \\ \vdots \\ k_t^\top \end{bmatrix}, \qquad V_t = \begin{bmatrix} v_1^\top \\ v_2^\top \\ \vdots \\ v_t^\top \end{bmatrix},$$

and ignoring forgetting for clarity, the ridge-regression solution is

$$S_t = \left(K_t^\top K_t + \lambda I\right)^{-1} K_t^\top V_t.$$

This introduces a key covariance matrix:

$$C_t = K_t^\top K_t + \lambda I.$$

The inverse C_t^{-1} is expensive to compute at every token, but it provides a more statistically informed memory readout.

Conjugate Gradient for Scalable Matrix Inverses

The lecture mentions recent work replacing explicit matrix inverse computation with conjugate gradient methods. To compute

$$x = C^{-1}b,$$

one solves

$$Cx = b$$

iteratively. Conjugate gradient is attractive when C is symmetric positive definite and matrix-vector products with C are cheaper than forming or inverting C explicitly.

A generic conjugate-gradient iteration is:

$$r_0 = b - Cx_0, \qquad p_0 = r_0,$$

$$\alpha_k = \frac{r_k^\top r_k}{p_k^\top C p_k},$$

$$x_{k+1} = x_k + \alpha_k p_k,$$

$$r_{k+1} = r_k - \alpha_k C p_k,$$

$$\beta_k = \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k},$$

$$p_{k+1} = r_{k+1} + \beta_k p_k.$$

The key systems question is whether these iterative solves can be integrated with chunkwise parallel training. The lecture states that this direction makes more advanced memory layers scalable for the first time.

Hardware-Level Explanation

Why the Naive Recurrent Form Is Slow

The recurrent linear attention kernel has the following token loop:

```
for t in range(L):
    S += outer(k[t], v[t])
    o[t] = q[t] @ S
```

For each token:

- The state S is $d \times d$.
- The update is a rank-one outer product.
- The readout is a matrix-vector multiply.

- The next iteration depends on the previous S.

This creates limited sequence-level parallelism. If B is small and the number of heads is modest, there may not be enough independent work to fill all SMs.

Rank-one updates and matrix-vector multiplies have lower arithmetic intensity than tiled GEMMs. They also do not map as cleanly onto tensor cores, especially when executed one token at a time.

Why Chunkwise Form Is GPU-Friendly

The chunkwise update

$$S_i = S_{i-1} + K_i^\top V_i$$

is a matrix multiplication:

$$(d \times C) \times (C \times d) \to (d \times d).$$

The historical readout

$$O_i^{\text{history}} = Q_i S_{i-1}$$

is also a matrix multiplication:

$$(C \times d) \times (d \times d) \to (C \times d).$$

The local attention term contains

$$Q_i K_i^\top,$$

which is

$$(C \times d) \times (d \times C) \to (C \times C).$$

Then

$$(Q_i K_i^\top) V_i$$

is

$$(C \times C) \times (C \times d) \to (C \times d).$$

All major steps can be expressed as GEMMs or small triangular operations. This is much better aligned with tensor cores.

Memory Hierarchy Behavior

A fused Flash Linear Attention kernel attempts to load a chunk of Q, K, V once, reuse it, and write final outputs. Conceptually:

1. Load Q_i, K_i, V_i tiles from HBM.
2. Keep relevant tiles in registers or shared memory.
3. Compute local attention contribution.
4. Compute state update contribution.
5. Compute historical readout contribution.
6. Write O_i and possibly chunk-boundary state.

The optimization objective is to avoid writing intermediate matrices such as

$$Q_i K_i^\top \in \mathbb{R}^{C \times C}$$

to HBM. Instead, the kernel can compute it tile by tile and immediately multiply by V_i.

Occupancy and Parallelism

For a fixed token budget T_{batch}, batch size scales approximately as

$$B = \frac{T_{\text{batch}}}{L}.$$

The number of independent batch-head tasks is

$$N_{\text{tasks}} = B \times H.$$

If

$$N_{\text{tasks}} \ll N_{\text{SM}},$$

then a purely recurrent implementation may underutilize the GPU. Chunkwise algorithms create additional parallel tasks across chunks:

$$N_{\text{chunk tasks}} = B \times H \times \frac{L}{C}.$$

This helps fill the GPU for long sequences.

Shared Memory Limitations

One question in the lecture discussion concerns programming models. Triton is easy for machine learning researchers but does not expose all low-level controls, such as detailed shared-memory management, in the same way as CUDA C++ or specialized frameworks. ThunderKittens and similar tools expose lower-level control, but often require C++ expertise. The speaker expresses interest in intermediate Python-like GPU programming systems that expose more control than Triton while remaining accessible.

Mathematical Reasoning

Softmax Attention Complexity

For one head:

$$QK^\top : (L \times d)(d \times L) \to L \times L.$$

The cost is

$$O(L^2 d).$$

Multiplying by V:

$$(L \times L)(L \times d) \to L \times d$$

also costs

$$O(L^2 d).$$

Thus total attention cost is

$$O(L^2 d).$$

Linear Attention Recurrent Complexity

For each token, updating

$$S_t = S_{t-1} + k_t v_t^\top$$

costs $O(d^2)$. Reading out

$$o_t = q_t^\top S_t$$

also costs $O(d^2)$. Across L tokens:

$$\text{Cost} = O(Ld^2).$$

Memory for state is

$$O(d^2).$$

Chunkwise Linear Attention Derivation

Let chunk i contain positions from a to $a + C - 1$. The full linear attention output at local position r is

$$o_{i,r} = q_{i,r}^\top \left(\sum_{j<a} k_j v_j^\top + \sum_{\ell=1}^{r} k_{i,\ell} v_{i,\ell}^\top \right).$$

Define

$$S_{i-1} = \sum_{j<a} k_j v_j^\top.$$

Then

$$o_{i,r} = q_{i,r}^\top S_{i-1} + \sum_{\ell=1}^{r} (q_{i,r}^\top k_{i,\ell}) v_{i,\ell}.$$

Stacking all r in the chunk gives

$$O_i = Q_i S_{i-1} + \left(Q_i K_i^\top \odot M_C \right) V_i.$$

This proves the chunkwise expression is exact.

Associative Retrieval Error

For $S = \sum_{j=1}^{L} k_j v_j^\top$, retrieving with k_m gives

$$k_m^\top S = \sum_{j=1}^{L} (k_m^\top k_j) v_j.$$

Separate desired and error terms:

$$k_m^\top S = (k_m^\top k_m)v_m + \sum_{j \neq m} (k_m^\top k_j)v_j.$$

If keys are normalized so that

$$\|k_j\|_2 = 1,$$

then the desired coefficient is 1, and the error is determined by pairwise key similarities:

$$e_m = \sum_{j \neq m} \cos(k_m, k_j)v_j.$$

When $L > d$, perfect orthogonality among all keys is impossible. Therefore, vanilla linear attention has an inherent retrieval-collision problem.

DeltaNet Gradient Derivation

The loss is

$$\mathcal{L}(S) = \frac{1}{2} \left\| S^\top k - v \right\|_2^2.$$

Let

$$r = S^\top k - v.$$

Then

$$\mathcal{L}(S) = \frac{1}{2} r^\top r.$$

The differential is

$$d\mathcal{L} = r^\top dr = r^\top d(S^\top k).$$

Since

$$d(S^\top k) = dS^\top k,$$

we have

$$d\mathcal{L} = r^\top dS^\top k = \mathrm{tr}(r^\top dS^\top k).$$

Using trace cyclicity,

$$d\mathcal{L} = \mathrm{tr}(kr^\top dS^\top) = \mathrm{tr}((kr^\top)^\top dS).$$

Therefore,

$$\nabla_S \mathcal{L} = kr^\top = k(S^\top k - v)^\top.$$

The gradient descent update is

$$S^+ = S - \beta k(S^\top k - v)^\top = S + \beta k(v - S^\top k)^\top.$$

DeltaNet Transition Matrix

Expanding the DeltaNet update:

$$S_t = S_{t-1} + \beta_t k_t v_t^\top - \beta_t k_t k_t^\top S_{t-1}.$$

Group the terms involving S_{t-1}:

$$S_t = \left(I - \beta_t k_t k_t^\top\right) S_{t-1} + \beta_t k_t v_t^\top.$$

This gives the structured transition matrix

$$A_t = I - \beta_t k_t k_t^\top.$$

The structure is crucial because it avoids arbitrary dense state transitions.

Code Walkthrough

Naive Recurrent Linear Attention

```
def linear_attention_recurrent(Q, K, V):
    # Q, K, V: [L, d]
    L, d = Q.shape
    S = zeros((d, d))
    O = zeros((L, d))

    for t in range(L):
        S += outer(K[t], V[t])
        O[t] = Q[t] @ S

    return O
```

This code directly implements

$$S_t = S_{t-1} + k_t v_t^\top, \qquad o_t = q_t^\top S_t.$$

Hardware implications:

- The loop over t is sequential.
- The outer product updates a $d \times d$ state.
- The readout is a matrix-vector product.
- Tensor cores are poorly utilized compared with GEMM.
- Parallelism must come from batch and head dimensions.

Chunkwise Linear Attention Pseudocode

```
def linear_attention_chunkwise(Q, K, V, C):
    # Q, K, V: [L, d]
    L, d = Q.shape
    S = zeros((d, d))
    O = zeros((L, d))

    for start in range(0, L, C):
        end = start + C

        Qi = Q[start:end]        # [C, d]
        Ki = K[start:end]        # [C, d]
        Vi = V[start:end]        # [C, d]

        # Contribution from previous chunks.
        O_history = Qi @ S       # [C, d]

        # Local causal linear attention inside the chunk.
        scores = Qi @ Ki.T       # [C, C]
        scores = causal_mask(scores)
        O_local = scores @ Vi    # [C, d]

        O[start:end] = O_history + O_local

        # Update chunk-boundary recurrent state.
        S = S + Ki.T @ Vi        # [d, d]

    return O
```

The line

$$S = S + K_i^\top V_i$$

is the central optimization: many rank-one updates become one GEMM.

Hardware implications:

- $Q_i S$, $Q_i K_i^\top$, $scores \cdot V_i$, and $K_i^\top V_i$ are matrix multiplications.
- The chunk size C should be chosen for GPU tile efficiency.
- A high-performance implementation should fuse operations and avoid materializing *scores* in HBM.

Naive DeltaNet Recurrence

```
def deltanet_recurrent(Q, K, V, beta):
    # Q, K, V: [L, d]
    # beta: [L]
    L, d = Q.shape
    S = zeros((d, d))
    O = zeros((L, d))

    for t in range(L):
        old_v = S.T @ K[t]
        delta_v = V[t] - old_v
        S = S + beta[t] * outer(K[t], delta_v)
        O[t] = Q[t] @ S

    return O
```

This implements:

$$\hat{v}_t = S_{t-1}^\top k_t,$$

$$S_t = S_{t-1} + \beta_t k_t (v_t - \hat{v}_t)^\top.$$

Hardware implications:

- The recurrence is sequential.
- Each step has a matrix-vector product and an outer product.
- It is more expressive than vanilla linear attention but harder to train efficiently.
- Chunkwise WY-like algorithms are needed to recover GPU efficiency.

Conceptual Fused Chunk Kernel

```
/*
Conceptual CUDA-style pseudocode.

Each program instance handles one batch, one head, and one chunk.
A real implementation would tile d and C, use tensor cores,
and carefully manage registers and shared memory.
*/

for each chunk i in parallel:
    load Q_i, K_i, V_i

    // Historical contribution.
    O_hist = matmul(Q_i, S_prev)

    // Local causal contribution.
    scores = matmul(Q_i, transpose(K_i))
    scores = apply_causal_mask(scores)
    O_local = matmul(scores, V_i)

    // Combine outputs.
    O_i = O_hist + O_local

    // Chunk state update.
    Delta_S = matmul(transpose(K_i), V_i)
    S_next = S_prev + Delta_S

    store O_i
    store S_next if needed
```

A production kernel would not literally store all intermediate matrices. Instead, it would tile computations so that partial products remain in registers or shared memory. The objective is to minimize HBM traffic.

Performance Intuition

Why Linear Attention Is Not Automatically Fast

The phrase **linear attention** can be misleading. Although the asymptotic sequence scaling is linear in the recurrent form, the naive implementation can be slow because it is sequential and tensor-core unfriendly.

The real performance question is not only:

$$\text{Does the algorithm scale as } O(L)?$$

It is also:

Does the algorithm expose enough parallel GEMM-shaped work to saturate the GPU?

Compute-Bound versus Memory-Bound Behavior

The chunkwise algorithm improves arithmetic intensity by reusing Q_i, K_i, V_i tiles. However, if intermediate states or local attention matrices are materialized to HBM, the algorithm may become memory-bound.

A fused implementation tries to keep the effective arithmetic intensity high:

Higher reuse $\Rightarrow$ fewer bytes per FLOP $\Rightarrow$ better chance of reaching tensor-core throughput.

Chunk Size Selection

The chunk size C controls a trade-off:

- Small C: more frequent recurrent updates, less local quadratic work, but smaller GEMMs and possibly lower tensor-core efficiency.
- Large C: larger GEMMs and better hardware utilization, but more local $O(C^2 d)$ work and less frequent state updates in some architectures.

The chunkwise linear attention complexity is

$$O(Ld^2) + O(LCd).$$

Thus, very large C increases the local attention cost. Practical values such as 64, 128, or 256 are chosen to match GPU tile sizes and memory constraints.

Why In-Context Retrieval Is Difficult for Recurrent Models

Softmax attention keeps the full KV cache:

$$\{(k_j, v_j)\}_{j=1}^{t}.$$

Linear recurrent models compress all history into a fixed-size state:

$$S_t \in \mathbb{R}^{d \times d}.$$

Compression is lossy when the number of associations is large. Retrieval errors arise from non-orthogonal keys. DeltaNet reduces this problem by learning an update that corrects previous predictions rather than blindly accumulating outer products.

Why Distillation Is Attractive

Training efficient architectures from scratch is expensive. Distilling a pretrained transformer into an efficient recurrent or hybrid architecture may reduce cost. However, the lecture notes that in-context retrieval can degrade even in distilled hybrid models. This matters for long chain-of-thought reasoning because the model must remember intermediate results.

A promising direction is to distill into architectures with stronger associative memory, such as DeltaNet-like or Mesa-like layers, rather than into weaker recurrent layers.

Common Misconceptions

Misconception: Linear Attention Is Just Faster Attention

Linear attention is not automatically faster. The recurrent form has linear sequence complexity, but naive implementations may underutilize GPUs. Practical speed requires chunkwise matrix multiplication and IO-aware fusion.

Misconception: Parallel Scan Solves Linear Attention Training

Parallel scan exposes sequence parallelism, but it may require materializing $O(Ld^2)$ states. This is often too memory intensive. It also does not automatically create tensor-core-friendly work.

Misconception: Removing Softmax Has No Modeling Cost

Removing softmax changes the memory mechanism. Softmax attention can directly retrieve from explicit KV storage. Linear attention compresses history into a fixed matrix state, creating retrieval collisions when many keys are not orthogonal.

Misconception: Bigger State Always Solves Retrieval

Increasing d increases state size d^2 and allows more nearly orthogonal keys. But memory management still matters. DeltaNet improves retrieval not only by having a matrix state, but by using an update rule that can erase and rewrite associations.

Misconception: Local Chunk Attention Makes the Method Approximate

For chunkwise linear attention, the decomposition into historical and local terms is exact:

$$O_i = Q_i S_{i-1} + (Q_i K_i^\top \odot M_C) V_i.$$

It is not an approximation of full causal linear attention. Approximation enters only if the architecture intentionally changes the update frequency or local mechanism.

Misconception: Triton Gives All Low-Level GPU Control

Triton is excellent for many ML kernels, but some low-level memory-management patterns are difficult or unavailable. Researchers building unusual sequence layers may need intermediate programming tools that combine Python usability with finer control over shared memory and scheduling.

Practical Takeaways

1. Linear attention replaces the KV cache with a fixed-size matrix-valued recurrent state:
$$S_t = \sum_{j=1}^{t} k_j v_j^\top.$$
2. The recurrent form is memory efficient for inference but not hardware efficient for training if implemented naively.
3. Chunkwise linear attention is the key transformation that converts rank-one updates into matrix multiplications.
4. The exact chunkwise output is:
$$O_i = Q_i S_{i-1} + (Q_i K_i^\top \odot M_C) V_i.$$
5. Flash Linear Attention is an IO-aware fused implementation of the chunkwise algorithm.
6. Vanilla linear attention has retrieval errors because keys are not perfectly orthogonal.
7. DeltaNet improves associative recall by performing an online regression update:
$$S_t = S_{t-1} + \beta_t k_t (v_t - S_{t-1}^\top k_t)^\top.$$
8. DeltaNet can be interpreted as gradient descent on a fast-weight linear regression objective.
9. More expressive test-time training layers use nonlinear fast-weight predictors, but their nonlinear recurrences are harder to parallelize.
10. Long-context performance depends on both modeling quality and GPU utilization; asymptotic complexity alone is not enough.

Project or Experiment Ideas

Benchmark Naive versus Chunkwise Linear Attention

Implement the naive recurrent form and chunkwise form in PyTorch. Measure runtime as a function of:

- sequence length L,
- head dimension d,
- chunk size C,
- batch size B.

Expected result: the recurrent implementation becomes increasingly unattractive for long sequences and larger d, while chunkwise computation benefits from GEMM.

Measure Retrieval Error

Generate random normalized keys and values. Store them using

$$S = \sum_{j=1}^{L} k_j v_j^\top.$$

Retrieve with each key:

$$\hat{v}_j = S^\top k_j.$$

Measure

$$\frac{1}{L} \sum_{j=1}^{L} \|\hat{v}_j - v_j\|_2^2$$

as a function of L/d. The error should grow as the number of stored associations exceeds the key dimension.

Implement DeltaNet Recurrence

Implement the simple recurrent DeltaNet update and compare associative recall against vanilla linear attention. Use synthetic key-value tasks where keys are random vectors and values are one-hot vectors or integers.

Profile With Nsight Compute

For a CUDA or Triton implementation, profile:

- achieved occupancy,
- tensor-core utilization,
- global memory throughput,
- shared memory usage,
- register pressure,
- achieved FLOP/s.

Compare naive recurrent kernels with chunkwise kernels.

Explore Chunk Size

Sweep

$$C \in \{16, 32, 64, 128, 256, 512\}$$

and measure throughput and memory usage. Identify where local $O(C^2 d)$ cost begins to dominate.

Hybrid Local Attention and Recurrent Memory

Implement a toy hybrid layer:

$$O_i = \text{softmax}\left(Q_i K_i^\top + M_C\right) V_i + Q_i S_{i-1}.$$

Compare it to pure linear attention on synthetic tasks requiring both local and long-range recall.

Visual Concept

Draw a long token sequence split into chunks of length C. Above each chunk, draw a local triangular attention matrix. Between chunks, draw a recurrent state S_i passed from left to right. Show that each chunk receives S_{i-1}, computes local attention, produces O_i, and emits S_i.

Visual Concept

Draw vanilla softmax attention as a large $L \times L$ matrix stored or streamed during training, and draw linear attention as a compact $d \times d$ memory matrix. Then

annotate the trade-off: softmax has explicit retrieval with large memory, while linear attention has compressed retrieval with possible interference.

Visual Concept

Draw the DeltaNet update as three steps: retrieve old value $\hat{v}_t = S_{t-1}^\top k_t$, compute residual $v_t - \hat{v}_t$, and write correction $\beta_t k_t (v_t - \hat{v}_t)^\top$ into the state.

Final Mental Model

Linear attention is best understood as **a recurrent associative memory implemented with a matrix-valued hidden state**. Its basic update accumulates key-value outer products, replacing the KV cache with a fixed-size fast memory:

$$S_t = S_{t-1} + k_t v_t^\top.$$

This solves the inference memory growth problem, but naive training is not GPU efficient because the recurrence is sequential and dominated by tensor-core-unfriendly operations. The key optimization is the chunkwise transformation:

$$O_i = Q_i S_{i-1} + (Q_i K_i^\top \odot M_C) V_i, \qquad S_i = S_{i-1} + K_i^\top V_i.$$

This turns many small recurrent operations into larger GEMM-like computations and enables Flash Linear Attention-style IO fusion.

The modeling weakness of vanilla linear attention is retrieval interference: many key-value pairs are compressed into a fixed $d \times d$ state, and non-orthogonal keys create cross terms. DeltaNet improves this by treating the recurrent state as a fast weight trained online:

$$S_t = S_{t-1} + \beta_t k_t (v_t - S_{t-1}^\top k_t)^\top.$$

This update retrieves, compares, and corrects memory. More advanced architectures such as Gated DeltaNet, TTT layers, and Mesa-like layers continue this idea: they make sequence models more expressive by turning recurrence into online learning, while using chunkwise algorithms to keep the computation compatible with modern GPU hardware.